

Memory Layer: Lokaler Memory-MCP-Server per Docker

Applied AI SS26 | HTWG Konstanz

In dieser Uebung setzt ihr euren eigenen **persistenten Memory Layer** auf.

Euer Agent (smolagents oder ein eigener) kann Fakten, Entscheidungen und Erkenntnisse speichern und spaeter per semantischer Suche wieder abrufen.

Das Ganze laeuft **komplett lokal** in einem Docker Container, ohne Cloud, ohne API Keys fuer die Vektordatenbank. Der Container nutzt SQLite-vec als Vektorstore und all-MiniLM-L6-v2 (ONNX) fuer Embeddings.

1. Docker installieren (falls noch nicht vorhanden)

Plattform	Download
Windows	Docker Desktop for Windows
macOS	Docker Desktop for Mac
Linux	Docker Engine

Nach der Installation: Docker Desktop starten und warten, bis das Icon in der Taskleiste "Docker Desktop is running" anzeigt.

2. Memory Container starten

Ein Befehl, alle Plattformen:

```
docker run -d --name memory -p 8765:8765 \
  -v memory_data:/app/sqlite_db \
  -e MCP_MODE=streamable-http \
  -e MCP_SSE_HOST=0.0.0.0 \
  -e MCP_SSE_PORT=8765 \
  -e MCP_ALLOW_ANONYMOUS_ACCESS=true \
  -e MCP_MEMORY_STORAGE_BACKEND=sqlite_vec \
  -e MCP_MEMORY_SQLITE_PATH=/app/sqlite_db/memory.db \
  doobidoo/mcp-memory-service:latest
```

Windows PowerShell (einzeilig, Copy-Paste-freundlich):

```
docker run -d --name memory -p 8765:8765 -v memory_data:/app/sqlite_db -e
MCP_MODE=streamable-http -e MCP_SSE_HOST=0.0.0.0 -e MCP_SSE_PORT=8765 -e
MCP_ALLOW_ANONYMOUS_ACCESS=true -e MCP_MEMORY_STORAGE_BACKEND=sqlite_vec -e
MCP_MEMORY_SQLITE_PATH=/app/sqlite_db/memory.db doobidoo/mcp-memory-service:latest
```

Wichtig: Nach dem ersten Start einmalig ein fehlendes Paket nachinstallieren und den Container neustarten:

```
docker exec memory pip install aiosqlite --quiet
docker restart memory
```

Der Container laeuft jetzt im Hintergrund und stellt einen MCP Server auf Port 8765 bereit.

3. Pruefen, ob es laeuft

```
docker logs memory
```

Ihr solltet eine Zeile sehen wie:

Streamable HTTP mode enabled on 0.0.0.0:8765

4. Python-Pakete installieren

```
pip install "smolagents[mcp]"
```

Falls ihr schon `smolagents` installiert habt, reicht:

```
pip install "smolagents[mcp]" --upgrade
```

5. Demo ausfuehren

```
python demo_smolagents.py
```

Das Skript verbindet sich per MCP mit eurem Container, speichert Fakten und sucht sie semantisch wieder. Wenn alles durchlauft, ist euer Memory Layer einsatzbereit.

Verfuegbare Memory-Tools

Tool	Beschreibung
<code>memory_store</code>	Fakt/Erkenntnis speichern (mit optionalen Tags)
<code>memory_search</code>	Semantische Suche ueber gespeicherte Memories
<code>memory_list</code>	Alle Memories auflisten
<code>memory_delete</code>	Memory loeschen (per Hash)

Tool	Beschreibung
memory_health	Healthcheck des Servers

Wie der Agent das nutzt

```
from smolagents import CodeAgent, MCPClient, InferenceClientModel

server = {"url": "http://localhost:8765/mcp", "transport": "streamable-http"}

with MCPClient(server) as tools:
    agent = CodeAgent(tools=tools, model=InferenceClientModel())
    agent.run("Speichere: Ich bevorzuge Python gegenueber Java.")
    agent.run("Was weisst du ueber meine Praeferenzen?")
```

Container stoppen und neustarten

```
docker stop memory      # Stoppt den Container
docker start memory     # Startet ihn wieder (Daten bleiben erhalten)
docker rm -f memory     # Loescht den Container komplett
```

Eure gespeicherten Memories bleiben erhalten, solange das Docker Volume `memory_data` existiert. Selbst wenn ihr den Container loescht und neu erstellt, sind die Daten noch da.

Fehlerbehebung

Problem	Loesung
docker: command not found	Docker ist nicht installiert oder nicht im PATH
Port 8765 belegt	Anderen Port nutzen: <code>-p 9999:8765</code> und URL in Demo anpassen
Container startet nicht	<code>docker logs memory</code> pruefen
MCPClient Timeout	Container braucht beim ersten Start etwas laenger (Modell wird geladen)